

Programação em Ambiente Web

Trabalho Prático - Grupo 13 | ESTG, P.Porto

Marketplace

Relatório – Milestone 1

Grupo 13

8240310 Daniel Filipe Cerqueira Queirós

8240307 Francisco Bernardo Morais de Azevedo

24 de abril de 2026

0 Conteúdo

1	Introdução	3
1.1	Objetivos da Plataforma	3
1.2	Perfis de Utilizador	3
2	Arquitetura e Tecnologias	4
2.1	Stack Tecnológica	4
2.2	Arquitetura MVC	4
2.3	Segurança	5
3	Funcionalidades Implementadas	5
3.1	Rotas de Interface (EJS)	5
3.1.1	Autenticação — /auth	5
3.1.2	Painel de Administração — /admin	5
3.1.3	Portal do Supermercado — /supermarket	6
3.1.4	Portal do Estafeta — /courier	6
3.2	API REST — /api	6
3.2.1	Utilizadores	7
3.2.2	Supermercados	7
3.2.3	Categorias e Produtos	7
3.2.4	Encomendas	8
3.2.5	Cupões	8
3.3	Fluxo de Estados de Encomenda	8
3.4	Notificações por Email	9
3.5	Funcionalidades Extra (Bonificação)	9
4	Como Colocar a Solução a Executar	10
4.1	Pré-requisitos	10
4.2	Instalação e Execução	10
4.3	Dependências Principais	11
5	Logins de Teste	11
5.1	Percursos de Teste Recomendado	11
5.1.1	Admin	11
5.1.2	Supermercado	12
5.1.3	Estafeta	12
6	Considerações Finais	12

1 Introdução

O presente relatório descreve o trabalho desenvolvido pelo Grupo 13 no âmbito da unidade curricular de **Programação em Ambiente Web (PAW)**. O projeto consiste no desenvolvimento de uma plataforma *marketplace* para supermercados locais, que centraliza a gestão de lojas, produtos, encomendas, estafetas e clientes através de uma aplicação web completa.

1.1 Objetivos da Plataforma

A plataforma oferece um conjunto alargado de funcionalidades, organizadas por perfil de utilizador:

- Registo e aprovação de supermercados com notificação por email;
- Gestão de produtos;
- Venda em caixa, cupões de desconto e método de entrega;
- Fluxo completo de encomendas com estados e notificações automáticas por email;
- Rede de estafetas para aceitação e conclusão de entregas ao domicílio;
- Sistema de cupões de desconto por percentagem, com validade e limite de usos;
- Mapa interativo com geolocalização automática dos supermercados;
- API REST completamente documentada com Swagger.

1.2 Perfis de Utilizador

A plataforma define quatro perfis com permissões diferenciadas e portais independentes:

Perfil	Responsabilidades
admin	Gestão global: aprovação de lojas, gestão de utilizadores, categorias, produtos, encomendas, cupões e mapa
supermarket	Gestão da sua loja: produtos, vendas presenciais, encomendas recebidas, cupões e definições
courier	Aceitação de entregas disponíveis e marcação como entregue
customer	Cliente final: registo e histórico de encomendas (M2)

2 Arquitetura e Tecnologias

2.1 Stack Tecnológica

Componente	Tecnologia / Versão
Runtime	Node.js $\geq$ 18
Framework web	Express 4
Base de dados	MongoDB + Mongoose 9
Motor de templates	EJS
Autenticação	JWT em cookie <code>HttpOnly</code> (<code>jsonwebtoken</code> 9)
Upload de ficheiros	Multer 2
Email	Nodemailer 8 (Gmail SMTP)
Segurança	<code>express-mongo-sanitize</code> (<code>helmet</code> removido temporariamente*)
Geocodificação	Nominatim (OpenStreetMap API)
Mapa interativo	Leaflet.js + OpenStreetMap
Documentação API	Swagger
UI / CSS	Bootstrap 5.3 + Bootstrap Icons 1.11
Alertas frontend	SweetAlert2

*O middleware `helmet` foi removido por incompatibilidade com o Leaflet.js; será reintegrado na Milestone 2.

2.2 Arquitetura MVC

O projeto segue uma arquitetura **MVC (Model-View-Controller)** com separação clara de responsabilidades:

- **Models** (`/models`): esquemas Mongoose para `User`, `Supermarket`, `Product`, `Category`, `Order` e `Coupon`;
- **Views** (`/views`): templates EJS organizados por perfil (`admin/`, `supermarket/`, `courier/`, `auth/`);
- **Controllers** (`/controllers`): lógica de negócio separada em controllers de API e controllers de vistas;
- **Routes** (`/routes`): mapeamento de rotas por domínio: `api.js`, `admin.js`, `supermarket.js`, `courier.js`;
- **Middlewares** (`/middlewares`): `verifyToken` (validação JWT), `verifyRole` (controlo de acesso por perfil) e `verifySupermarketStatus` (bloqueia supermercados não aprovados);
- **Utils** (`/utils`): camada de serviços que encapsula toda a lógica de acesso à base de dados, isolando os controllers das operações Mongoose; cada entidade tem o seu serviço dedicado (`userService`, `supermarketService`, `productService`, `categoryService`, `orderService`, `couponService`), complementados por `emailService`, `swagger.js` e `seed.js` (inicialização da conta superadmin).

2.3 Segurança

- Tokens JWT armazenados em cookies `HttpOnly`, inacessíveis via JavaScript no browser;
- Passwords com hash `bcrypt` (salt rounds: 10);
- `express-mongo-sanitize` para prevenção de injeção NoSQL;
- Controlo de acesso por role em todas as rotas protegidas;
- Validação de propriedade em operações sensíveis (ex: cancelamento de encomenda);
- **Conta superadmin** protegida: criada automaticamente na primeira inicialização via `utils/seed.js`, com flag `isSuperAdmin` na base de dados; é invisível na tabela de admins, imune a edição e eliminação por qualquer utilizador. Credenciais por defeito: `superadmin@gmail.com / estg2026`;
- **Nota:** o middleware `helmet` foi removido por incompatibilidade com o mapa interativo `Leaflet.js` (os seus cabeçalhos de segurança bloqueavam o carregamento dos tiles `OpenStreetMap`). A sua reintegração compatível com o mapa está prevista para a Milestone 2.

3 Funcionalidades Implementadas

3.1 Rotas de Interface (EJS)

3.1.1 Autenticação — /auth

Método	Rota	Descrição
GET	/auth/login	Página de login
POST	/auth/login	Submissão de credenciais
GET	/auth/logout	Logout (elimina cookie JWT)
GET	/auth/register	Página de registo de supermercado
POST	/auth/register	Submissão de registo

3.1.2 Painel de Administração — /admin

Todas as rotas requerem perfil `admin`.

Rota	Descrição
/admin/dashboard	Dashboard com estatísticas globais (lojas, utilizadores, encomendas)
/admin/approvals	Aprovação e rejeição de supermercados pendentes
/admin/users	Gestão de todos os utilizadores (tabs por perfil)
/admin/categories	Criação e gestão de categorias de produtos
/admin/products	Visão global de todos os produtos
/admin/product/:id	Detalhe e edição de produto
/admin/orders	Listagem de todas as encomendas
/admin/order/:id	Detalhe de encomenda com histórico
/admin/supermarket/:id	Detalhe de supermercado com estatísticas
/admin/user/:id	Detalhe de utilizador
/admin/map	Mapa interativo com supermercados georreferenciados
/admin/coupons	Listagem de todos os cupões da plataforma
/admin/coupon/:id	Detalhe e edição de cupão

3.1.3 Portal do Supermercado — /supermarket

Todas as rotas requerem perfil `supermarket` e estado `approved`.

Rota	Descrição
/supermarket/dashboard	Dashboard com métricas e encomendas recentes
/supermarket/settings	Definições da loja (nome, morada, horário, entregas) e conta
/supermarket/products	Gestão de produtos (criar, editar, ativar/desativar)
/supermarket/product/:id	Detalhe e edição de produto
/supermarket/orders	Listagem e gestão de encomendas recebidas
/supermarket/order/:id	Detalhe de encomenda e progressão de estado
/supermarket/newsale	Registar nova venda presencial com carrinho
/supermarket/coupons	Criação e gestão de cupões da loja

3.1.4 Portal do Estafeta — /courier

Todas as rotas requerem perfil `courier`.

Rota	Descrição
/courier/dashboard	Dashboard com entrega ativa (se existir)
/courier/deliveries	Lista de entregas disponíveis para aceitar
/courier/history	Histórico de entregas concluídas
/courier/order/:id	Detalhe da entrega e ação de conclusão

3.2 API REST — /api

A API devolve sempre JSON. A autenticação é feita via cookie `token` (JWT HttpOnly). A documentação interativa completa está disponível em `/api-docs` (Swagger UI).

3.2.1 Utilizadores

M	Rota	Acesso	Descrição
POST	/api/users/createAdmin	admin	Criar admin
GET	/api/users/listAdmins	admin	Listar admins
GET	/api/users/admin/:id	admin	Detalhe de admin
PUT	/api/users/updateAdmin/:id	admin	Editar admin
DELETE	/api/users/deleteAdmin/:id	admin	Eliminar admin
POST	/api/users/createCourier	admin	Criar estafeta
GET	/api/users/listCouriers	admin	Listar estafetas
GET	/api/users/courier/:id	admin	Detalhe de estafeta
PUT	/api/users/updateCourier/:id	admin	Editar estafeta
DELETE	/api/users/deleteCourier/:id	admin	Eliminar estafeta
POST	/api/customers/create	admin, supermarket	Criar cliente
GET	/api/customers/list	admin	Listar clientes
GET	/api/customers/:id	admin	Detalhe de cliente
GET	/api/customers/email/:email	admin, supermarket	Cliente por email
PUT	/api/customers/update/:id	admin	Editar cliente
DELETE	/api/customers/delete/:id	admin	Eliminar cliente

3.2.2 Supermercados

M	Rota	Acesso	Descrição
POST	/api/supermarkets/create	admin	Criar supermercado
GET	/api/supermarkets/list	admin	Listar todos
GET	/api/supermarkets/listPending	admin	Listar pendentes
GET	/api/supermarkets/map	público	Dados para o mapa
GET	/api/supermarkets/:id	admin	Detalhe
PUT	/api/supermarkets/approve/:id	admin	Aprovar
PUT	/api/supermarkets/reject/:id	admin	Rejeitar
PUT	/api/supermarkets/update/:id	admin	Editar (admin)
PUT	/api/supermarkets/me	supermarket	Editar a própria loja
DELETE	/api/supermarkets/delete/:id	admin	Eliminar

3.2.3 Categorias e Produtos

M	Rota	Acesso	Descrição
POST	/api/categories/create	admin	Criar categoria
GET	/api/categories/list	admin, supermarket	Listar
GET	/api/categories/:id	admin	Detalhe
PUT	/api/categories/update/:id	admin	Editar
DELETE	/api/categories/delete/:id	admin	Eliminar
POST	/api/product/create	admin, supermarket	Criar produto
GET	/api/product/list/me	supermarket	Produtos da minha loja
GET	/api/product/list/:id	admin, supermarket	Produtos por loja
GET	/api/product/:id	admin, supermarket	Detalhe
PUT	/api/product/update/:id	admin, supermarket	Editar
PUT	/api/product/toggle/:id	admin, supermarket	Ativar/desativar
DELETE	/api/product/delete/:id	admin, supermarket	Eliminar

3.2.4 Encomendas

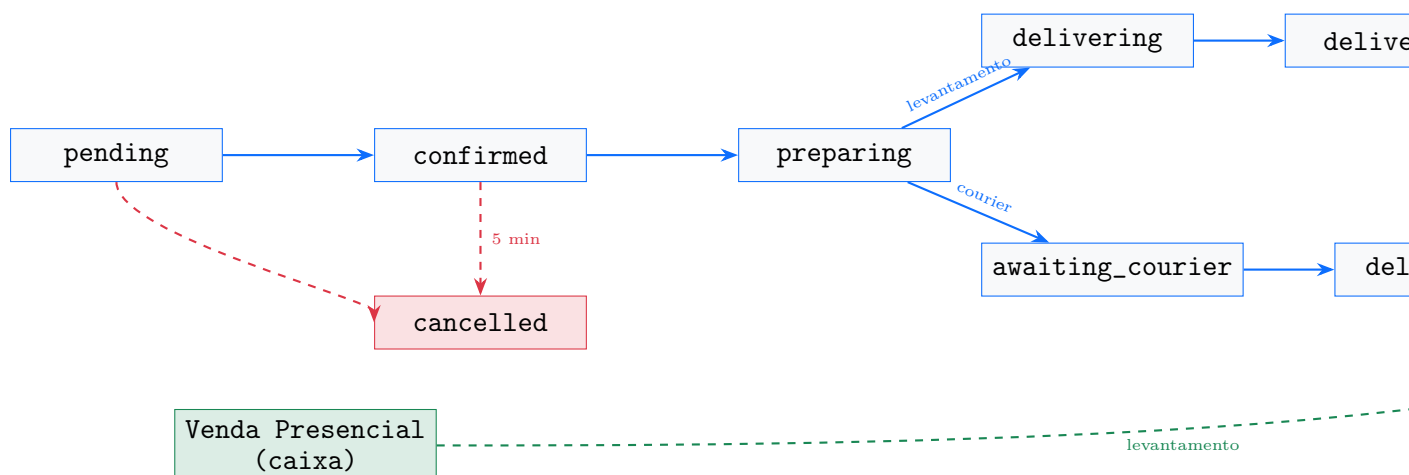
M	Rota	Acesso	Descrição
POST	/api/orders/sale	supermarket	Venda presencial
GET	/api/orders/list	admin	Todas as encomendas
GET	/api/orders/supermarket/:id	admin, supermarket	Por supermercado
GET	/api/orders/customer/:id	admin, supermarket	Por cliente
GET	/api/orders/:id	admin, supermarket	Detalhe
PUT	/api/orders/status/:id	admin, supermarket	Atualizar estado
PUT	/api/orders/cancel/:id	admin, supermarket	Cancelar
PUT	/api/orders/courier/accept/:id	courier	Aceitar entrega
PUT	/api/orders/courier/complete/:id	courier	Concluir entrega

3.2.5 Cupões

M	Rota	Acesso	Descrição
POST	/api/coupons/create	supermarket	Criar cupão
GET	/api/coupons/list	admin, supermarket	Listar
GET	/api/coupons/:id	admin, supermarket	Detalhe
GET	/api/coupons/validate	supermarket	Validar cupão
PUT	/api/coupons/update/:id	admin, supermarket	Editar
PUT	/api/coupons/toggle/:id	admin, supermarket	Ativar/desativar
DELETE	/api/coupons/delete/:id	admin, supermarket	Eliminar

3.3 Fluxo de Estados de Encomenda

O ciclo de vida de uma encomenda passa pelos seguintes estados:



O cancelamento repõe automaticamente o **stock** de todos os produtos. Para encomendas no estado **confirmed**, o cancelamento só é permitido nos primeiros **5 minutos**.

Nota: as vendas presenciais registadas em caixa (`/supermarket/newsale`) com método de levantamento ficam diretamente no estado **delivered**, sem percorrer os estados intermédios. Se o método for **courier**, a venda entra no estado **confirmed** e segue o fluxo normal.

3.4 Notificações por Email

São enviadas notificações automáticas via Nodemailer (Gmail SMTP) nos seguintes eventos:

Evento	Destinatário	Trigger
Mudança de estado de encomenda	Cliente	Qualquer transição de estado
Aprovação / rejeição de registo	Supermercado	Ação do admin
Nova entrega disponível	Todos os estafetas livres	Estado awaiting_courier

3.5 Funcionalidades Extra (Bonificação)

As funcionalidades abaixo constituem extras implementados para além do enunciado base, elegíveis para bonificação.

Mapa Interativo

A página `/admin/map` apresenta todos os supermercados aprovados georreferenciados, utilizando **Leaflet.js** com tiles OpenStreetMap. As coordenadas são obtidas automaticamente via **Nominatim** (geocodificação reversa) no momento do registo ou edição da morada, sem intervenção manual.

Cupões de Desconto

Sistema completo de cupões com as seguintes características:

- Criados pelo **supermercado** e válidos apenas na sua loja;

- Apenas desconto por percentagem (1%–100%);
- Controlo de data de expiração e número máximo de usos;
- Ativação/desativação pelo supermercado e pelo admin;
- O admin pode visualizar e editar todos os cupões da plataforma, com informação do supermercado criador;
- Aplicados em vendas presenciais (/supermarket/newsale) com desconto calculado em tempo real.

Notificações por Email

Descrito na secção anterior: envio automático em todos os eventos relevantes do ciclo de vida de encomendas e supermercados.

API Documentada com Swagger

Documentação interativa completa em /api-docs com OpenAPI 3.0: todos os endpoints com esquemas de request/response, exemplos e autenticação via cookie JWT.

4 Como Colocar a Solução a Executar

4.1 Pré-requisitos

- Node.js ≥ 18 ;
- MongoDB local ou MongoDB Atlas;
- Conta Gmail com *App Password* para envio de emails (opcional).

4.2 Instalação e Execução

1. Instalar as dependências:

```
npm install
```

2. Criar o ficheiro .env na raiz do projeto (o projeto já vai com .env lá dentro configurado):

```
MONGO_URI=mongodb+srv://<user>:<pass>@<cluster>.mongodb.net
/<db>
secret=<chave-secreta-jwt>
EMAIL_USER=<email@gmail.com>
EMAIL_PASS=<app-password-gmail>
```

3. Iniciar o servidor em modo de desenvolvimento:

```
npm start
```

4. Aceder à aplicação em <http://localhost:3000>. A documentação Swagger está em <http://localhost:3000/api-docs>.

4.3 Dependências Principais

Pacote	Versão	Utilização
express	~4.16	Framework web
mongoose	^9.3	ODM MongoDB
jsonwebtoken	^9.0	Autenticação JWT
bcrypt	^6.0	Hash de passwords
ejs	~2.6	Motor de templates
multer	^2.1	Upload de imagens
nodemailer	^8.0	Envio de emails
express-mongo-sanitize	^2.2	Segurança NoSQL
swagger-ui-express	^5.0	Documentação API
swagger-jsdoc	^6.2	Geração OpenAPI
dotenv	^17	Variáveis de ambiente

5 Logins de Teste

Os seguintes utilizadores estão disponíveis na base de dados de teste: AINDA VOU CRIAR

Perfil	Email	Password	Notas
Superadmin	superadmin@gmail.com	estg2026	Criado automaticamente; não editável no
Admin	admin@marketplace.pt	admin123	Acesso total
Supermercado	mercado@marketplace.pt	super123	Estado: aprovado
Estafeta	estafeta@marketplace.pt	courier123	Disponível
Cliente	cliente@marketplace.pt	cliente123	Conta ativa

Nota: A conta `superadmin@gmail.com` (password: `estg2026`) é criada automaticamente na primeira inicialização do servidor via `utils/seed.js`; não é necessário inserção manual. As restantes credenciais devem ser substituídas pelas contas reais presentes na base de dados de teste antes da entrega.

5.1 Percursos de Teste Recomendado

5.1.1 Admin

1. Login em `/auth/login` com conta admin;
2. Verificar o dashboard (`/admin/dashboard`) com contagens globais;
3. Aprovar um supermercado pendente em `/admin/approvals` verificar email de aprovação;
4. Criar um estafeta e um cliente em `/admin/users`;
5. Criar uma categoria de produto em `/admin/categories`;
6. Consultar os cupões da plataforma em `/admin/coupons` e ver o detalhe de um;
7. Visualizar o mapa de supermercados em `/admin/map`;

8. Testar a API em `/api-docs` (fazer login primeiro para o cookie estar ativo).

5.1.2 Supermercado

1. Login com conta de supermercado aprovado;
2. Configurar a loja em `/supermarket/settings` (morada, horário, métodos de entrega);
3. Adicionar um produto em `/supermarket/products` com imagem e categoria;
4. Registrar uma venda presencial em `/supermarket/newsale`: pesquisar cliente por email, adicionar produtos, aplicar cupão (opcional), finalizar;
5. Gerir o estado da encomenda criada em `/supermarket/orders`;
6. Criar um cupão da loja em `/supermarket/coupons`.

5.1.3 Estafeta

1. Login com conta de estafeta;
2. Verificar entregas disponíveis em `/courier/deliveries` (requer encomenda no estado `awaiting_courier`);
3. Aceitar a entrega e confirmá-la em `/courier/order/:id`;
4. Consultar o histórico de entregas em `/courier/history`.

6 Considerações Finais

Na Milestone 1 foram implementadas as funcionalidades base da plataforma, com foco no **backoffice**: gestão interna pelos perfis *admin*, *supermarket* e *courier*. A API REST está completa e documentada com Swagger, o sistema de autenticação com JWT em cookie garante separação de acessos entre perfis, e os extras de bonificação (mapa georreferenciado, cupões de desconto e notificações por email) estão totalmente implementados.

Para a **Milestone 2** está previsto o desenvolvimento do frontoffice para o cliente final: navegação no catálogo de supermercados, encomenda online com carrinho e cupões, e histórico de encomendas com cancelamento. O endpoint `PUT /api/orders/cancel/:id` está implementado e funcional, mas a sua utilização pela interface do cliente (botão de cancelar encomenda) será integrada apenas na Milestone 2, no contexto do portal do cliente.